Relatório de Auditoria de Segurança — Lecionário

Isolamento de tenant, permissões, IDOR, segredos e inputs sem tratamento

DATA
2026-09-08

REPOSITÓRIO
github.com/rilsonjoas/lecionario

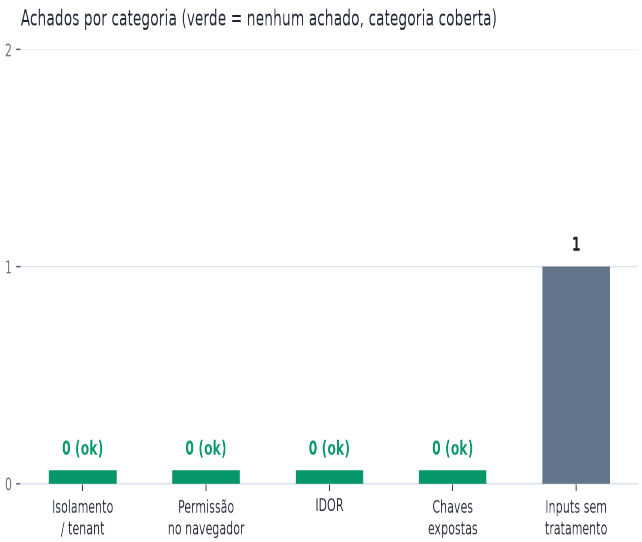
ESCOPO AUDITADO
Monorepo lecionario (lecionario-web: Next.js App Router + React Query; lecionario-mobile: Expo/React Native). Escopo: todas as rotas de src/app (4 páginas), contexts (FavoritesContext, web e mobile), scripts de build/conteúdo (export-daily-refs.ts, cálculo de calendário litúrgico), Dockerfile/docker-compose.yml, e superfície de renderização de conteúdo devocional/liturgia.

NOTA METODOLÓGICA
Mecanismo de isolamento de tenant/RLS: **não se aplica** — confirmado que o projeto não tem backend próprio, banco de dados, API routes (0 arquivos route.ts em src/app), login ou conta de usuário nenhuma. Todo conteúdo (leituras, devocionais, citações) é estático, bundlado no build, 100% autoral. "Permissão no navegador" e "IDOR" também não se aplicam pelo mesmo motivo — sem servidor que aplique ou deixe de aplicar uma permissão, sem recurso endereçável por ID que pertença a alguém. O único estado que existe (favoritos) fica inteiramente no dispositivo (localStorage no web, AsyncStorage no mobile), nunca sincronizado com um servidor — não há "outro usuário" pra vazar dado de. "Chaves expostas" checado ainda assim (Sentry DSN, scripts de build, Dockerfile). XSS mapeado pra stack real (Next.js web + RN mobile): dangerouslySetInnerHTML, libs de markdown/HTML, WebView, eval/new Function.

1 achado — 1 Informativa — **nenhuma vulnerabilidade explorável encontrada**

Resumo executivo

Auditoria limpa. Das 5 categorias verificadas em código real, 4 não se aplicam ou não tiveram achado: o projeto confirmadamente não tem backend, banco de dados, autenticação, nem qualquer recurso endereçável por ID que pertença a um usuário — é um site/app de conteúdo estático. O único item registrado (L1) é **informativo/housekeeping**: código morto que carrega um padrão sensível mas não é explorável, e nem tem como vir a ser, dado que não existe fonte de dado dinâmico no projeto. Nada aqui bloqueia produção nem exige correção urgente.



Pontos fortes (verificados, com evidência)

✓ **Ausência real de superfície de ataque server-side** — Confirmado sistematicamente: 0 arquivos route.ts/API em src/app, 0 ocorrências de signIn/login/session/auth fora de uma citação de texto (lewis-quotes.ts), 0 variáveis de ambiente necessárias em produção (.env.example: "Nenhuma variável de ambiente necessária. Todos os dados são locais"). Não é ausência de verificação — é ausência real de backend.

✓ **Favoritos isolados por dispositivo, nunca sincronizados** — FavoritesContext (web e mobile) lê/escreve só em localStorage/AsyncStorage locais — nenhuma chamada de rede, nenhum servidor envolvido. Sem conceito de "outro usuário" pra um IDOR ou vazamento cross-tenant existir aqui.

✓ **Falha de storage tratada sem quebrar a UI** — toggleFavorite e a hidratação inicial (FavoritesContext.tsx) envolvem localStorage/JSON.parse em try/catch — navegador com storage bloqueado (modo privado, política restritiva) degrada pra "sem favoritos", não pra crash.

✓ **Hidratação SSR-safe** — A leitura de localStorage roda só depois do mount (useEffect), com comentário explícito sobre não divergir do HTML gerado no servidor — evita o erro clássico de hydration mismatch do Next.js, que teria sido um bug de robustez, não de segurança, mas mostra o mesmo cuidado.

✓ **Sentry DSN público documentado como intencional** — O DSN commitado no docker-compose.yml vem com o comentário explícito de que é credencial pública por design (vai pro bundle do cliente de qualquer forma) — padrão recomendado pela própria Sentry, não um segredo vazado por engano.

✓ **Sem WebView, eval ou new Function em nenhum dos dois apps** — Busca em lecionario-web e lecionario-mobile por WebView/eval(/new Function(não encontrou nenhuma ocorrência — toda renderização é JSX puro (React escapa texto por padrão) ou React Native puro (sem superfície de HTML nenhuma).

✓ **Scripts de build isolados do runtime de produção** — scripts/export-daily-refs.ts e scripts/test-liturgia-parser.ts (o único uso de cheerio no projeto) rodam manualmente, offline, sobre arquivo local — não são parte do processo que serve o site pros usuários, então não são superfície de ataque em produção.

Pontos fracos

Nenhum risco de segurança identificado. O item L1 é hardening preventivo de código morto, de prioridade mínima dado que este projeto não tem — e não tem planos de ter — nenhuma fonte de dado dinâmico que pudesse algum dia torná-lo explorável.

Achados detalhados

Informativa

L1 · Inputs sem tratamento (equiv. XSS) / housekeeping

`lecionario-web/src/components/ui/chart.tsx:77–95`

dangerouslySetInnerHTML em componente shadcn não utilizado

ChartStyle usa dangerouslySetInnerHTML pra gerar um <style> a partir de config.theme/config.color — mesmo boilerplate padrão do shadcn/ui encontrado nos outros projetos da mesma stack (instalado via CLI, nunca importado de fato). Busca por <Chart em todo lecionario-web/src/app e lecionario-web/src/components não encontrou nenhum uso. Sem exploração possível hoje: não há dado dinâmico (de conteúdo ou de qualquer input) alimentando esse config em lugar nenhum do código atual — todo o conteúdo do site é estático, bundlado no build.

```
// lecionario-web/src/components/ui/chart.tsx
<style
dangerouslySetInnerHTML={{
__html: Object.entries(THEMES).map(([theme, prefix]) => `
${prefix} [data-chart=${id}] { ... ${itemConfig.color} ... }
`).join('\n'),
}}
/>
// nunca importado: grep por "<Chart" em src/app e src/components → 0 resultados
```

Impacto: Nenhum hoje. Se o componente Chart for adotado no futuro com config vindo de dado dinâmico, o dangerouslySetInnerHTML atual injetaria isso sem sanitização numa tag <style> sem revisão adicional — é um código morto com um padrão sensível embutido, não uma vulnerabilidade ativa. Risco prático ainda menor que nos outros projetos da mesma stack, porque este site não tem NENHUMA fonte de dado dinâmico (nem admin, nem API, nem submissão pública) que pudesse um dia alimentar esse config.

Correção sugerida: Remover o componente Chart não utilizado (e o import do pacote recharts associado, se não usado em outro lugar) — reduz superfície de código que precisaria ser reauditado se algum dia for religado por engano. Baixíssima prioridade dado o contexto (site 100% estático).

Categorias sem achado (verificadas explicitamente)

✓ **1 — Isolamento de tenant / RLS** — Não se aplica: 0 arquivos route.ts/API, 0 banco de dados, 0 conta de usuário. Não há dado multi-tenant nenhum pra isolar.

✓ **2 — Permissão definida no navegador** — Não se aplica: sem servidor, não existe um par frontend×backend de permissão pra comparar. Nenhuma tela/ação é restrita por papel.

✓ **3 — IDOR** — Não se aplica: nenhuma rota (nem página, nem API) recebe um ID de recurso que pertença a um usuário. O único estado (favoritos) é local ao dispositivo, nunca endereçado por ID num servidor.

✓ **4 — Chaves expostas** — Verificado apesar de não haver backend: nenhuma variável de ambiente necessária em produção, único valor commitado é o DSN público do Sentry (documentado como intencional).

Recomendações priorizadas

Prior.	Ação	Por quê
P1	(opcional, esforço mínimo) Remover componente shadcn Chart não utilizado (L1)	chart.tsx nunca é importado e não tem nenhuma fonte de dado dinâmico no projeto pra alimentá-lo — risco teórico, mas remover código morto com dangerouslySetInnerHTML é limpeza de baixo custo.

Sem P2/P3 — não há mais nenhuma ação de segurança pendente identificada nesta auditoria.

Issues para o GitHub

Texto completo em Markdown, pronto para copiar e colar. Só 1 issue aqui, de housekeeping — não há achado de segurança acionável que justifique uma issue de correção urgente.

--- ISSUE 1 ---

[Housekeeping] Remover componente shadcn Chart não utilizado (dangerouslySetInnerHTML morto)

Labels: security, severidade:informativa, housekeeping

```
**Problema**

`lecionario-web/src/components/ui/chart.tsx:77-95` — `ChartStyle` usa `dangerouslySetInnerHTML` pra gerar
um `<style>` a partir de `config.theme`/`config.color`. Boilerplate padrão do shadcn/ui, instalado mas
**nunca importado** (`grep "<Chart"` em todo `src/app` e `src/components` → 0 resultados).

**Por que vale remover**

Não é explorável hoje — este site não tem NENHUMA fonte de dado dinâmico (sem backend, sem admin, sem
submissão pública, sem API routes) que pudesse alimentar `config` com algo controlável por alguém além do
próprio autor. Mas é código morto com um padrão sensível (`dangerouslySetInnerHTML`) que passaria
despercebido numa reintrodução futura (ex.: alguém copia um exemplo do shadcn docs que usa `<Chart>` com
dado vindo de uma fonte nova).

**Evidência**

```tsx
<style
dangerouslySetInnerHTML={{
__html: Object.entries(THEMES).map(([theme, prefix]) => `...`).join('\n'),
}}
/>
```

Impacto

Nenhum hoje. Risco é só de reintrodução futura sem revisão, e mesmo esse risco é baixo dado que o projeto
não tem plano de adicionar backend.

Sugestão de correção

Remover `lecionario-web/src/components/ui/chart.tsx` (e o import do pacote `recharts` associado, se não
usado em outro lugar) caso nenhum gráfico esteja planejado no roadmap próximo.

Critérios de aceite

- [ ] `chart.tsx` removido ou sua geração de CSS deixa de usar `dangerouslySetInnerHTML`.
- [ ] Build e testes de `lecionario-web` continuam passando sem ele.
```

--- FIM ISSUE 1 ---